Can Test-Time Memory Evolution Outperform Test-Time Training for Scientific Discovery?

Yepeng Liu*,† UC Santa Barbara
Shengmai Chen* UC Santa Barbara
Xuandong Zhao UC Berkeley
Chengzhi Liu UC Santa Barbara
Dawn Song UC Berkeley
Xin Eric Wang UC Santa Barbara
Yuheng Bu UC Santa Barbara

Abstract

Recent *test-time training* (TTT) frameworks have enabled open-weight large language models (LLMs) to outperform inference-only frameworks powered by frontier models on scientific discovery tasks. However, TTT introduces additional computational cost for online weight updates during the discovery process, making computational efficiency an important consideration when scaling to larger models. This motivates us to explore an efficient *test-time adaptation* method for scientific discovery. We introduce *Test-Time Memory Evolution* (TTME), a plug-and-play test-time adaptation method, as an alternative to TTT. TTME continually updates the external memory by distilling the experience to guide subsequent discovery without updating model weights. To have a clean comparison, we evaluate TTME and TTT under the same discovery framework, replacing only the test-time adaptation method. Our evaluation shows that TTME outperforms TTT on all five mathematical tasks, while reducing computational cost by 82.8-84.6% in FLOPs and 47.7-58.4% in runtime. These results establish TTME as an effective and computationally efficient alternative to TTT in the evaluated discovery settings. Our code is available at https://github.com/yepengliu/TTME.

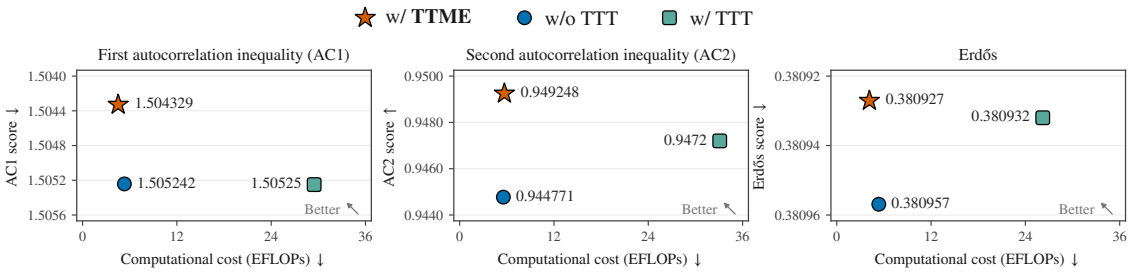


Figure 1: Best-found scores versus computational cost under the TTT-Discover framework [1] with Qwen3-8B. The three settings use the same discovery framework, with TTT, with TTME replacing TTT, or with neither method. ***TTME* achieves the best scores at low computational cost across all three tasks.**

1 Introduction

Scientific discovery often involves proposing hypotheses, evaluation, and using the resulting evidence to guide further investigation [2, 3]. Advances in reasoning, coding, and agentic tool-use capabilities of large language models (LLMs) have enabled systems that automate part of this discovery process [4, 5]. In the iterative discovery loop, a failed attempt can reveal an implementation constraint, while a

*These authors contributed equally to this work.
†Correspondence to: yepengliu@ucsb.edu.

successful modification can suggest an idea worth applying elsewhere. A key design question is how to learn from this accumulated experience to improve subsequent search efficiently.

Recent *test-time training* (TTT) frameworks, such as TTT-Discover [1] and ThetaEvolve [6], provide a promising solution to this challenge by combining iterative search with test-time reinforcement learning (RL) [7, 8]. Specifically, an LLM generates candidate solutions and receives evaluation feedback. These evaluated attempts provide training data for RL, which internalizes search experience in model weights to guide subsequent generation. Their results demonstrate that TTT is a powerful *test-time adaptation* method on challenging scientific discovery tasks, enabling smaller open-weight models to outperform inference-only systems based on frontier models, such as AlphaEvolve [4]. However, TTT requires online model weight adaptation during the whole discovery process, which is usually computationally expensive and time-consuming (see Table 3). It makes TTT hard to scale to larger models for more complex discovery tasks. This motivates us to explore: *under the same discovery framework, is there a more efficient test-time adaptation method that can achieve comparable or even better performance than test-time training?*

We hypothesize that evolving external memory could be a efficient alternative to online weight updates for scientific discovery. Our core intuition is that current LLMs already have broad foundational knowledge, reasoning and coding capabilities to use experience accumulated during discovery. Continually distilling and organizing this experience in external memory may therefore be sufficient to improve subsequent discovery, without internalizing it through weight updates. Building on this intuition, we introduce Test-Time Memory Evolution (TTME), a plug-and-play *self-evolving memory* system for scientific discovery. We update the memory from evaluation feedback and provide relevant content in the context to guide the subsequent steps.

Specifically, we divide the search experience into two components: practical lessons for implementing candidate solutions and algorithmic ideas for improving task scores. Correspondingly, TTME maintains two complementary memory modules: *Guideline Memory* and *Strategy Memory*. Guideline Memory distills execution and evaluation feedback into shared operational guidelines for reliable implementation. Strategy Memory extracts effective algorithmic ideas from score-improving modifications and consolidates related experiences into reusable strategies. We select and adapt relevant strategies to guide subsequent exploration, combining ideas from different search attempts.

Our contributions are summarized as follows:

- We investigate whether a test-time memory evolution can outperform test-time weight updates in scientific discovery, and introduce an plug-and-play self-evolving memory method that continually updates external memory from evaluated search experience to guide subsequent discovery with a fixed-weight model.
- We evaluate TTME under two TTT frameworks across five mathematical construction tasks and three models spanning 8B and 120B parameters. *We preserve each framework's experimental configuration, replacing only its TTT part with TTME. TTME outperforms the reported TTT results in all evaluated tasks.* Our analysis shows that TTME reduces running time by $47.7 - 58.4\%$ and computational costs by $82.8 - 84.6\%$ relative to TTT. These results demonstrate that TTME is a more efficient alternative to TTT for scientific discovery.
- We analyze the search behavior of TTME and TTT during the discovery. Both methods improve the implementation validity rate relative to the underlying discovery framework without TTT or TTME, but TTME achieves better performance. Moreover, compared with TTT, TTME more frequently builds on score-improving combinations of previously explored ideas.

2 Related Work

Non-Parametric Adaptation. Non-parametric discovery methods use fixed-weight LLMs to generate executable candidates and guide subsequent search with evaluation feedback [9, 10, 5, 11–13]. The effectiveness of this process depends on how search policies interact with the operators used to generate and modify candidates [14, 15]. FunSearch [9] and AlphaEvolve [4, 16] evolve candidate programs through repeated generation and evaluation with proprietary LLMs. Adaptive search methods further

use observed progress to adjust exploration intensity and allocate computation across candidate populations [17]. These studies establish search organization as an important factor in the quality and efficiency of automated discovery. Beyond selecting promising candidates, execution experience can also help subsequent exploration [18–24]. [25] and [26] compare candidate solutions and their evaluation results to identify promising modifications. [27] uses execution feedback to guide which optimizations to attempt next by adding, reprioritizing, or pruning proposed modifications in a search tree. These studies improve discovery through the joint design of search, planning, and feedback utilization. We instead study experience accumulation specifically as a modular adaptation mechanism and introduce TTME, a plug-and-play memory system for existing discovery frameworks.

Parametric Adaptation. Parametric methods update the LLMs through RL on evaluated search attempts [28–30]. ThetaEvolve [6] turns evolutionary program search into a dynamic RL environment. As its program archive evolves, training proceeds from newly discovered candidates rather than repeatedly starting from the same initial program. TTT-Discover [1] designs both its learning objective and search rule around finding a single high-quality solution rather than maximizing average performance. It exponentially upweights high-reward attempts during training and uses the best observed outcomes to prioritize states for further search. These methods internalize search experience in model weights, enabling open-weight models to surpass results achieved with proprietary models [6, 1]. However, continually updating model weights during discovery incurs additional computational cost. We show that evolving external memory offers a more effective and efficient alternative to online weight adaptation. TTME achieves better discovery outcomes within both frameworks, and our efficiency experiments demonstrate lower computational cost.

3 Method

3.1 Background and Problem Formulation

We study scientific discovery as an iterative process of proposing hypotheses, evaluating them, and using the resulting evidence to guide subsequent proposals. We focus on tasks with executable candidate programs and a task-specific verifier, where a program produces a construction whose validity and objective value are assessed by the verifier.

Let $v(x) \in \{0, 1\}$ indicate whether candidate program x produces a valid result, and let $s(x)$ denote its evaluated score, oriented so that larger values are better. For the set $\mathcal{X}_{\text{eval}}$ of candidates evaluated within the search budget, the best valid score is

$$s^\star = \max_{\substack{x \in \mathcal{X}_{\text{eval}} \\ v(x)=1}} s(x). \tag{1}$$

Our goal is to find the highest-scoring valid candidate within a given search budget.

Our study builds on two existing TTT scientific discovery frameworks, TTT-Discover [1] and ThetaEvolve [6]. We describe two frameworks using a common formulation. Each framework maintains an archive of search states, each representing a candidate solution and its associated records. At each search step, the framework selects states from the archive as parents for new attempts. The code associated with a selected parent is its parent program. Given the task specification, parent program, and associated records, the LLM generates candidate programs. We execute and evaluate these programs, recording their validity, scores, and execution feedback. The framework then updates the archive for subsequent search. We preserve each framework's original parent-selection and archive-update rules.

Both frameworks use search feedback to update model weights through test-time RL. *We ask whether non-parametric methods, e.g., memory adaptation, can substitute for these parametric methods, e.g., online weight adaptation.* Writing $\mathcal{D}_t$ for evaluated search records available to an update, we contrast two adaptation mechanisms:

$$\begin{aligned} &\text{Weight adaptation:} && \theta_{t+1} \leftarrow U_{\text{RL}}(\theta_t, \mathcal{D}_t), \\ &\text{Memory adaptation:} && \mathcal{M}_{t+1} \leftarrow U_{\text{mem}}(\mathcal{M}_t, \mathcal{D}_t), \qquad \theta_t = \theta_0. \end{aligned} \tag{2}$$

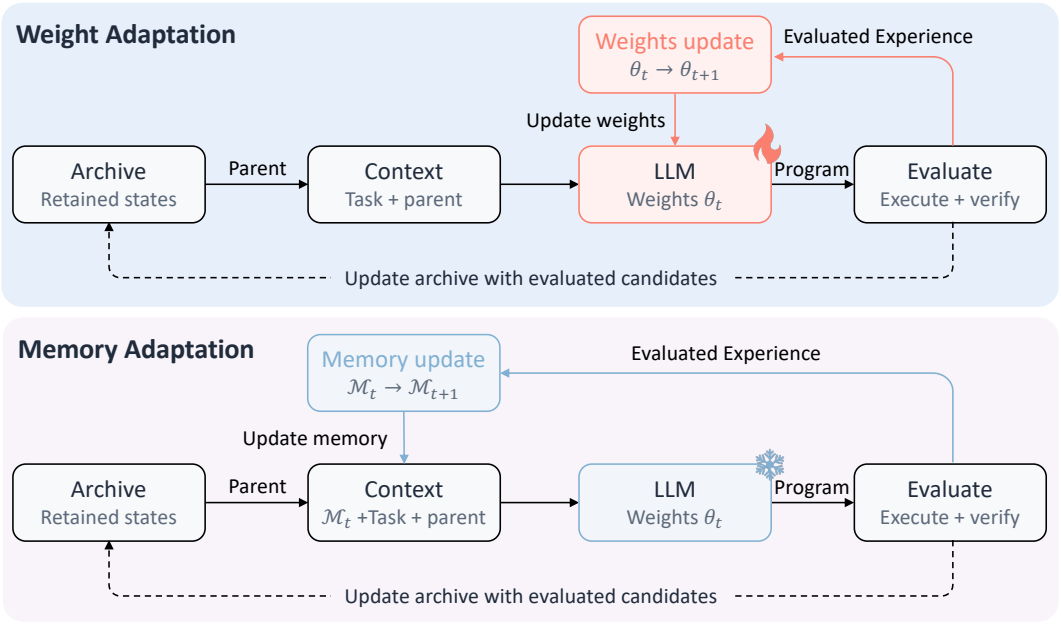


Figure 2: Weight adaptation updates model parameters through test-time RL (top). Memory adaptation updates external memory that conditions generation while keeping model weights fixed (bottom). Both retain the same underlying discovery framework.

Here, U_{RL} updates model parameters, whereas U_{mem} updates external memory used to construct subsequent generation contexts. t denotes the search step, θ_t is the model parameter, and $\mathcal{M}_t$ is the external memory. We retain the underlying discovery framework and task verifier while replacing only TTT with TTME. Figure 2 compares these two adaptation mechanisms.

Evaluated search attempts provide practical lessons for implementing candidate solutions and algorithmic ideas for improving task scores. We organize this experience into two complementary memories, $\mathcal{M}_t = (\mathcal{L}_t, \mathcal{S}_t)$. Guideline Memory $\mathcal{L}_t$ stores shared guidelines for implementing candidate solutions. Strategy Memory $\mathcal{S}_t$ organizes algorithmic ideas into reusable strategies that we select and adapt to guide subsequent search.

3.2 Guideline Memory: Shared Operational Lessons

Execution and evaluation feedback can reveal implementation problems and choices that improve task scores. Lessons from one attempt can guide the implementation of later candidates. We distill these observations into compact guidelines stored in $\mathcal{L}_t$. Each guideline specifies when it applies, explains the relevant failure or improvement, and recommends a concrete action.

Every K search steps, we form a summary r_t of outcomes collected across parents since the previous update, including valid results, their scores, and observed failures. We update the memory in two stages, first proposing guidelines $\mathcal{G}_t$ and then incorporating them into the existing memory:

$$\mathcal{G}_t \leftarrow \mathrm{Distill}(r_t, \mathcal{L}_t), \qquad \mathcal{L}_{t+1} \leftarrow \mathrm{Maintain}(\mathcal{L}_t, \mathcal{G}_t, t). \tag{3}$$

Here, Distill prompts the same base model with r_t and the existing guidelines $\mathcal{L}_t$ to propose $\mathcal{G}_t$. In Maintain, we match each proposal against existing entries. For a match, we increment the entry's recurrence count and refresh its last reaffirmed step to t; otherwise, we add a new entry. We remove stale entries and enforce a capacity limit, prioritizing recurrence counts and breaking ties by recency. The retained guidelines provide shared context for implementing subsequent candidate solutions.

3.3 Strategy Memory: Reusable Algorithmic Experience

Combining existing ideas is important in scientific discovery. An idea behind a score-improving modification may also be useful for other candidates. Strategy Memory preserves these ideas and how

they were applied, so they can be adapted and combined in subsequent searches.

We update Strategy Memory every K search steps, together with Guideline Memory. We select evaluated cases where a modified program produces a valid result with a higher score than its parent, and denote this collection by $\mathcal{D}_t^+$. For each case, we analyze the original and modified programs, their differences, and the evaluation results using the same base model. The model extracts one or more *experience notes* from each case, each describing an algorithmic idea, how it was applied, and its source case. We denote the resulting collection of notes by $\mathcal{E}_t$.

Different modifications can implement the same algorithmic idea in different ways. We group experience notes that share an underlying idea into a *strategy family* and maintain a persistent *strategy entry* for each family. Each entry describes the shared idea, its application conditions, and the different ways it has been implemented, with references to the source cases. The extraction of experience notes and the update of Strategy Memory are expressed as:

$$\mathcal{E}_t \leftarrow \mathrm{Extract}(\mathcal{D}_t^+), \qquad \mathcal{S}_{t+1} \leftarrow \mathrm{Consolidate}(\mathcal{S}_t, \mathcal{E}_t). \tag{4}$$

Here, Extract denotes the case-wise extraction described above. In Consolidate, we use the same base model to compare new notes with existing entries and assign them to existing or new families. For an existing family, the model combines the new notes with the current entry to update its content. For a new family, the model creates an entry from its notes.

3.4 Strategy-Guided Search and Candidate Generation

With two memories established, we now describe how they guide new candidate generation. Guideline Memory provides shared guidance for implementation. Strategy Memory contains ideas drawn from specific modifications, which may need adaptation before they can be applied to another parent. We therefore propose *Strategy-Guided Search* to select relevant strategies and turn them into parent-specific modification instructions. When no suitable strategy is selected, we use the parent context and Strategy Memory to propose a new idea and formulate its modification instruction.

Specifically, at search step t, let $p_i^{(t)}$ denote the program associated with the i-th selected parent. Let $C_i^{(t)}$ denote the input context supplied by the underlying framework for the i-th parent at step t, such as task specification and parent program $p_i^{(t)}$.

For each parent, we use a *selector* (base model) to analyze and select up to N distinct strategies from Strategy Memory based on their potential to improve the parent program. The selector is provided with the parent context, a summary of each strategy entry, and the strategies already assigned to earlier parents in the current step. Selection proceeds sequentially across parents:

$$\mathbf{a}_i^{(t)} = \mathrm{Select}\big(C_i^{(t)}, \hat{\mathcal{S}}_t, \mathbf{A}_{<i}^{(t)}\big), \qquad \mathbf{a}_i^{(t)} = (a_{i,1}^{(t)}, \ldots, a_{i,N}^{(t)}), \tag{5}$$

Here, $\hat{\mathcal{S}}_t$ denotes the summaries of strategy entries. We collect the nonempty strategy selections for earlier parents into

$$\mathbf{A}_{<i}^{(t)} = \left\{ a_{j,n}^{(t)} \;\middle|\; 1 \le j < i,\ 1 \le n \le N,\ a_{j,n}^{(t)} \neq \varnothing \right\}.$$

Each nonempty $a_{i,n}^{(t)}$ denotes a selected strategy. If fewer than N strategies are selected, we set the remaining elements of $\mathbf{a}_i^{(t)}$ to $\varnothing$. The selector prioritizes improvement potential for the current parent. Among similarly plausible choices, it prefers a strategy not yet included in $\mathbf{A}_{<i}^{(t)}$.

Then, we produce a parent-specific modification instruction $d_{i,n}^{(t)}$ for $n = 1, \ldots, N$:

$$d_{i,n}^{(t)} = \begin{cases} \mathrm{Adapt}\big(C_i^{(t)}, \mathcal{S}_t[a_{i,n}^{(t)}]\big), & a_{i,n}^{(t)} \neq \varnothing, \\ \mathrm{Propose}\big(C_i^{(t)}, \mathcal{S}_t\big), & a_{i,n}^{(t)} = \varnothing. \end{cases} \tag{6}$$

For a selected strategy, one *adaptor* (base model) reads its full entry and the full parent code, then specifies how to apply the strategy to that parent. When $a_{i,n}^{(t)} = \varnothing$, the adaptor uses the parent and available strategy memory content to propose a new idea.

Algorithm 1 Scientific Discovery with Test-Time Memory Evolution

Require: Base model, task verifier, search budget, B candidates per parent, N modification instructions per parent, memory update interval $K \geq 1$, Direct Generation fraction $\alpha \in [0, 1]$.

Initialize archive, evaluated history, and memories $\mathcal{L}_0, \mathcal{S}_0$
for each search step $t = 0, 1, \ldots$ within the budget **do**
  Sample parents using a selection rule from the underlying framework
  for each parent i, sequentially **do**
    Select up to N distinct strategies using Eq. (5)
  end for
  for each parent i **do**
    Form N modification instructions using Eq. (6)
    Generate αB direct candidates using Eq. (7)
    Generate $(1 - \alpha)B/N$ candidates per instruction using Eq. (8)
  end for
  Evaluate candidates; record outcomes and commit archive updates
  if $(t + 1)$ mod $K = 0$ **then**
    Update $\mathcal{L}_{t+1}$ using Eq. (3)
    Update $\mathcal{S}_{t+1}$ using Eq. (4)
  else
    $\mathcal{L}_{t+1} \leftarrow \mathcal{L}_t$, $\mathcal{S}_{t+1} \leftarrow \mathcal{S}_t$
  end if
end for
return Best valid result found under the task objective

We use the constructed instructions to guide candidate generation. Adapting an explored strategy allows us to combine an idea extracted from a score-improving modification with the parent program's existing approach. Moreover, to explore additional ideas that may improve the task score, we also generate candidates without a modification instruction.

Specifically, let B denote the total candidate budget per parent. Both generation branches receive the parent context $C_i^{(t)}$ and Guideline Memory $\mathcal{L}_t$. We generate αB candidates through *Direct Generation* using this shared context, where α denotes the fraction of the budget allocated to this branch. We set $\alpha = 1/2$ in our experiments. For *Strategy-Guided Generation*, we additionally supply a modification instruction $d_{i,n}^{(t)}$ and generate $(1 - \alpha)B/N$ candidates for each of the N instructions:

$$x \leftarrow \text{Generate}(C_i^{(t)}, \mathcal{L}_t), \tag{7}$$

$$x \leftarrow \text{Generate}(C_i^{(t)}, \mathcal{L}_t, d_{i,n}^{(t)}). \tag{8}$$

The candidates' evaluation outcomes inform subsequent memory updates, which provide guidance for later search steps. The complete procedure is summarized in Algorithm 1. Implementation details and prompts appear in Appendices C and D.

4 Experiments

4.1 Experiment Settings

Models. We use Qwen3-8B [31] and gpt-oss-120b [32] in the TTT-Discover framework. For ThetaEvolve, we use DeepSeek-R1-0528-Qwen3-8B [8], abbreviated as Distill-Qwen3-8B. In our method, we keep the model weights fixed throughout the search. The same base model performs candidate generation and all memory-related operations.

Frameworks. We evaluate our method within two frameworks, TTT-Discover [1] and ThetaEvolve [6], that combine iterative search with test-time RL. We compare three settings within two frameworks: w/o TTT, which keeps model weights fixed without our TTME; w/ TTT, which updates model weights through test-time RL; and w/ TTME, which keeps model weights fixed and updates self-evolving

Table 1: Results under the TTT-Discover framework for three settings: w/ TTT, w/o TTT, and w/ TTME. Results for w/ TTT are from Yuksekgonul et al. [1] over 50 search steps.

Method	Model	Erdős ↓	AC1 ↓	AC2 ↑
TTT-Discover (w/ TTT)	Qwen3-8B	0.380932	1.50525	0.9472
TTT-Discover (w/o TTT)	Qwen3-8B	0.380957	1.505242	0.944771
Ours (w/ TTME)	Qwen3-8B	**0.380927**	**1.504329**	**0.949248**
TTT-Discover (w/ TTT)	gpt-oss-120b	0.380876	1.50287	0.9591
TTT-Discover (w/o TTT)	gpt-oss-120b	0.380945	1.505966	0.953682
Ours (w/ TTME)	gpt-oss-120b	**0.380862**	**1.502745**	**0.960784**

Table 2: Results under the ThetaEvolve framework for three settings: w/ TTT, w/o TTT, and w/ TTME. Results for w/ TTT are from Wang et al. [6] over 65 search steps.

Method	Model	AC2 ↑	AC3 ↓	Hadamard ↑
ThetaEvolve (w/ TTT)	Distill-Qwen3-8B	0.9469	1.4930	0.5764
ThetaEvolve (w/o TTT)	Distill-Qwen3-8B	0.947375	1.519277	0.571085
Ours (w/ TTME)	Distill-Qwen3-8B	**0.948148**	**1.464224**	**0.579938**

memory. For weight adaptation, we report results from the original papers, with model weights updated through test-time RL. Without weight adaptation, we run the underlying search procedure with fixed model weights and its existing generation context. Our method also keeps the weights fixed, but updates Guideline Memory and Strategy Memory to guide subsequent generation. We retain each framework's parent-selection rules, archive-update rules, and task verifier (settings in Appendix B).

Tasks. We evaluate five mathematical construction tasks. (1) `Erdős`: Erdős's minimum overlap problem searches for density functions that certify tighter upper bounds on the minimum-overlap constant. (2) `First autocorrelation inequality (AC1)`: AC1 constructs a nonnegative function f to minimize $\|f * f\|_\infty / \|f\|_1^2$, yielding an upper bound on the first autocorrelation constant. Here, $f * f$ denotes the autoconvolution of f. (3) `Second autocorrelation inequality (AC2)`: AC2 constructs a nonnegative function f to maximize $\|f * f\|_2^2 / (\|f * f\|_1 \|f * f\|_\infty)$, yielding a lower bound on the second autocorrelation constant. (4) `Third autocorrelation inequality (AC3)`: AC3 allows functions with both positive and negative values and seeks a tighter upper bound using the absolute autoconvolution peak. (5) `Maximal determinant problem (Hadamard)`: Hadamard maximizes the normalized absolute determinant of a 29×29 matrix with entries in $\{-1, +1\}$.

4.2 Main Results

Performance across frameworks and models. TTME outperforms both TTT and search without TTT across two discovery frameworks, three models, and five tasks (Tables 1 and 2). The comparison with TTT shows that evolving external memory can yield better discovery results than existing weight-update methods. The comparison with search without TTT shows the benefit of adding memory to the underlying fixed-weight search procedure. Together, these results establish TTME as an effective alternative to test-time weight adaptation across the evaluated settings. Appendix E visualizes the discovered constructions.

Progress during search. We next examine how TTME improves the best-found score over the course of search (Figure 3). Under the TTT-Discover framework, TTME surpasses the TTT scores within 25 steps across all three tasks and both models. Under the ThetaEvolve framework, TTME also surpasses the TTT scores on three tasks. These trajectories show that TTME can improve on the final TTT results before exhausting the search budget.

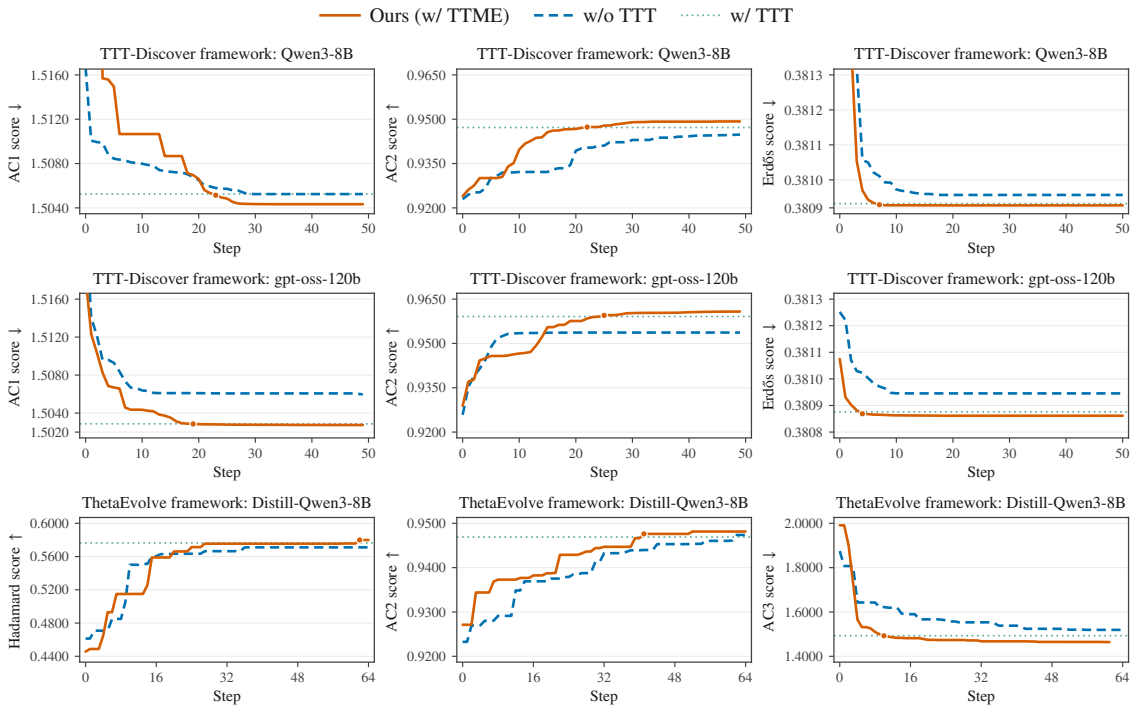


Figure 3: Best-found scores during search under TTT-Discover and ThetaEvolve frameworks. TTME achieves better final scores in every evaluated setting. Orange dots mark the first step at which TTME exceeds the corresponding final TTT score.

Table 3: Runtime (excluding evaluation) and computation under TTT-Discover with Qwen3-8B. TTME reduces runtime by 47.7–58.4% and FLOPs by 82.8–84.6%.

		Runtime excluding evaluation (h)			Computational cost (EFLOPs)		
Task	Model	TTT	TTME	Reduction	TTT	TTME	Reduction
AC1	Qwen3-8B	11.76	**5.36**	54.5%	29.47	**4.53**	84.6%
AC2	Qwen3-8B	12.73	**6.66**	47.7%	33.09	**5.69**	82.8%
Erdős	Qwen3-8B	11.67	**4.85**	58.4%	26.21	**4.15**	84.2%

Runtime and computational cost. We compare runtime (excluding evaluation) and computational cost under TTT-Discover with Qwen3-8B (Table 3). We include all candidate generation, memory-related LLM calls, and RL training where applicable. Across the three tasks, even though TTT-Discover updates only LoRA parameters [33], TTME reduces runtime by 47.7–58.4% and FLOPs by 82.8–84.6% relative to TTT. Combined with the performance results, these reductions show that TTME achieves better discovery outcomes with less runtime and computation.

4.3 Ablation Studies

Combining TTME with TTT. Having established the effectiveness of TTME with fixed model weights, we examine whether adding TTT further improves performance. We compare four settings under the TTT-Discover framework with Qwen3-8B: neither TTT nor TTME, TTT alone, TTME alone, and their combination (Table 4). In the combined setting, the RL-updated model performs both candidate generation and memory-related operations. TTME alone achieves the best scores on AC2 and Erdős, while adding TTT worsens both results. This comparison further supports using TTME as an alternative to TTT rather than requiring both forms of adaptation.

Contributions of the two memories. We examine the contributions of Guideline Memory and Strategy Memory under the TTT-Discover framework with gpt-oss-120b (Table 5). We compare full

Table 4: Comparison of TTT, TTME, and their combination against a baseline with neither. TTME alone achieves the best scores on both tasks; adding TTT reduces performance.

TTT	TTME	AC2 ↑	Erdős ↓
TTT-Discover framework: Qwen3-8B			
✗	✗	0.944771	0.380957
✓	✗	0.9472	0.380932
✗	✓	**0.949248**	**0.380927**
✓	✓	0.944141	0.380944

Table 5: Ablation of Guideline Memory and Strategy Memory. Full TTME achieves the best scores on all three tasks, demonstrating the benefit of combining both memory components.

Configuration	AC1 ↓	AC2 ↑	Erdős ↓
TTT-Discover framework: gpt-oss-120b			
Full TTME	**1.502745**	**0.960784**	**0.380862**
w/o Guideline Memory	1.505644	0.958506	0.380865
w/o Strategy Memory	1.505981	0.956493	0.380864
w/o TTME	1.505966	0.953682	0.380946

TTME with variants that remove either memory component or both. Full TTME achieves the best scores on all three tasks, and removing either component worsens performance. These results support retaining both shared operational lessons and reusable algorithmic strategies to guide discovery.

4.4 Search Behavior Analysis

Candidate validity and discovery performance. Successful discovery requires candidates that are both valid and capable of improving the task score. We consider a program valid if it compiles successfully and receives a score from the evaluator. Figure 4 compares mean validity over 50 steps and best-found scores under the TTT-Discover framework with Qwen3-8B. Both TTT and TTME improve the mean program validity rate over search without TTT. On AC1 and Erdős, TTT and TTME achieve similar validity rates, while TTME achieves better best-found scores. This suggests TTME generates more effective ideas for improving task scores.

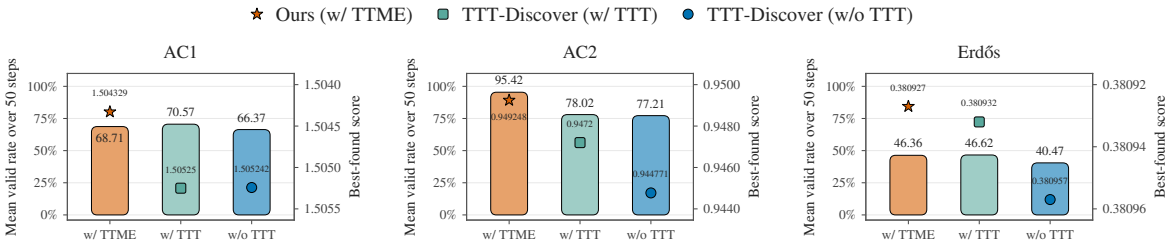


Figure 4: Implementation validity rate and final performance under the TTT-Discover framework with Qwen3-8B across three tasks. Bars show mean validity rates over 50 steps (left axes); markers show best-found scores (right axes), with better scores plotted higher.

Combining ideas for further search. Beyond validity, we examine how search combines and builds on previously explored ideas under the TTT-Discover framework with Qwen3-8B. We count a candidate as a combination if it retains an idea from its parent and adds an idea previously explored outside its ancestry. The added idea must be absent from the parent. We measure how often these combinations are selected as parents for further search.

TTME selects combinations more frequently than TTT on both AC1 (10.2% versus 8.2%) and AC2 (24.7% versus 18.6%; Figure 5). We also count selections of combinations that improve on their own parents by more than 10^{-5}. TTME again has higher shares on AC1 (5.4% versus 4.8%) and AC2 (17.1% versus 13.3%). Thus, TTME not only selects combinations of existing ideas more frequently for further

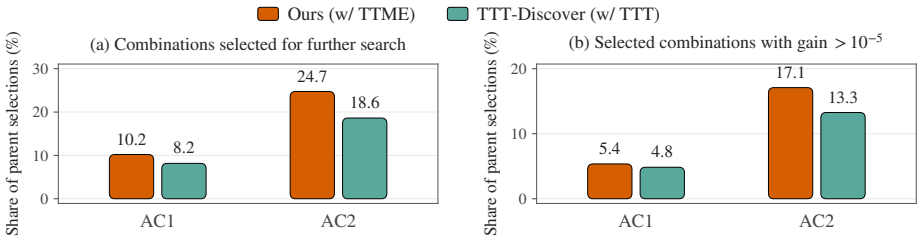


Figure 5: Combination of ideas in subsequent search under TTT-Discover with Qwen3-8B. TTME has a higher combination share on both tasks, with and without the 10^{-5} improvement requirement.

search, but also more frequently selects combinations that improve task scores. These observations support using external memory to combine ideas from different search branches and carry productive combinations into subsequent exploration.

5 Conclusion

We investigate whether evolving external memory can outperform test-time training for scientific discovery within the same discovery frameworks. We introduce TTME, which distills search experience into operational lessons and algorithmic strategies to guide a fixed-weight model. Across two frameworks, three models, and five mathematical tasks, TTME achieves better scores than TTT in every evaluated setting. Under TTT-Discover with Qwen3-8B, it reduces runtime excluding evaluation by 47.7–58.4% and FLOPs by 82.8–84.6%, including all memory-related calls. These results establish self-evolving memory as an effective and computationally efficient alternative to test-time weight updates for the discovery tasks we study.

References

[1] Mert Yuksekgonul, Daniel Koceja, Xinhao Li, Federico Bianchi, Jed McCaleb, Xiaolong Wang, Jan Kautz, Yejin Choi, James Zou, Carlos Guestrin, and Yu Sun. Learning to discover at test time. *arXiv preprint arXiv:2601.16175*, 2026.

[2] Ross D. King, Jem Rowland, Stephen G. Oliver, Michael Young, Wayne Aubrey, Emma Byrne, Maria Liakata, Magdalena Markham, Pinar Pir, Larisa N. Soldatova, Andrew Sparkes, Kenneth E. Whelan, and Amanda Clare. The automation of science. *Science*, 324(5923):85–89, 2009. doi: 10.1126/science.1165620. URL https://doi.org/10.1126/science.1165620.

[3] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI Scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024. URL https://arxiv.org/abs/2408.06292.

[4] Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025. URL https://arxiv.org/abs/2506.13131.

[5] Robert Tjarko Lange, Yuki Imajuku, and Edoardo Cetin. ShinkaEvolve: Towards open-ended and sample-efficient program evolution. *arXiv preprint arXiv:2509.19349*, 2025. URL https://arxiv.org/abs/2509.19349.

[6] Yiping Wang, Shao-Rong Su, Zhiyuan Zeng, Eva Xu, Liliang Ren, Xinyu Yang, Zeyi Huang, Xuehai He, Luyao Ma, Baolin Peng, Hao Cheng, Pengcheng He, Weizhu Chen, Shuohang Wang, Simon Shaolei Du, and Yelong Shen. ThetaEvolve: Test-time learning on open problems. *arXiv preprint arXiv:2511.23473*, 2025.

[7] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024. URL https://arxiv.org/abs/2402.03300.

[8] DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning, 2025. URL https://arxiv.org/abs/2501.12948.

[9] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024. doi: 10.1038/s41586-023-06924-6. URL https://www.nature.com/articles/s41586-023-06924-6.

[10] Chenglei Si, Zitong Yang, Yejin Choi, Emmanuel Candès, Diyi Yang, and Tatsunori Hashimoto. Towards execution-grounded automated ai research. *arXiv preprint arXiv:2601.14525*, 2026.

[11] Yepeng Liu, Yu Huang, Yu-Xiang Wang, Yingbin Liang, and Yuheng Bu. Convexbench: Can llms recognize convex functions? *arXiv preprint arXiv:2602.01075*, 2026.

[12] Yepeng Liu, Dasen Dai, Chengzhi Liu, Yiren Song, Hai Ci, Yu Zhang, Qi Zhang, Mike Zheng Shou, Xin Eric Wang, and Yuheng Bu. Figures as programs: Recursive generation of editable scientific figures. *arXiv preprint arXiv:2609.01006*, 2026.

[13] Fei Liu, Rui Zhang, Zhuoliang Xie, Rui Sun, Kai Li, Xi Lin, Zhenkun Wang, Zhichao Lu, and Qingfu Zhang. LLM4AD: A platform for algorithm design with large language model. *arXiv preprint arXiv:2412.17287*, 2024. URL https://arxiv.org/abs/2412.17287v1.

[14] Edan Toledo, Karen Hambardzumyan, Martin Josifoski, Rishi Hazra, Nicolas Baldwin, Alexis Audran-Reiss, Michael Kuchnik, Despoina Magka, Minqi Jiang, Alisia Lupidi, et al. Ai research agents for machine learning: Search, exploration, and generalization in mle-bench. *Advances in Neural Information Processing Systems*, 38:35309–35348, 2026.

[15] Terry Chen, Zhifan Ye, Bing Xu, Zihao Ye, Timmy Liu, Ali Hassani, Tianqi Chen, Andrew Kerr, Haicheng Wu, Yang Xu, et al. Avo: Agentic variation operators for autonomous evolutionary search. *arXiv preprint arXiv:2603.24517*, 2026.

[16] Bogdan Georgiev, Javier Gómez-Serrano, Terence Tao, and Adam Zsolt Wagner. Mathematical exploration and discovery at scale. *arXiv preprint arXiv:2511.02864*, 2025. URL https://arxiv.org/abs/2511.02864.

[17] Mert Cemri, Shubham Agrawal, Akshat Gupta, Shu Liu, Audrey Cheng, Qiuyang Mang, Ashwin Naren, Lutfi Eren Erdogan, Koushik Sen, Matei Zaharia, et al. Adaevolve: Adaptive llm driven zeroth-order optimization. *arXiv preprint arXiv:2602.20133*, 2026.

[18] Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J. Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. GEPA: Reflective prompt evolution can outperform reinforcement learning. In *International Conference on Learning Representations*, 2026. URL https://arxiv.org/abs/2507.19457.

[19] Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. Agentic context engineering: Evolving contexts for self-improving language models. In *International Conference on Learning Representations*, 2026. URL https://arxiv.org/abs/2510.04618.

[20] Yougang Lyu, Xi Zhang, Xinhao Yi, Yuyue Zhao, Shuyu Guo, Wenxiang Hu, Jan Piotrowski, Jakub Kaliski, Jacopo Urbani, Zaiqiao Meng, Lun Zhou, and Xiaohui Yan. EvoScientist: Towards multi-agent evolving AI scientists for end-to-end scientific discovery. *arXiv preprint arXiv:2603.08127*, 2026. URL https://arxiv.org/abs/2603.08127.

[21] Aofan Liu, Shiyuan Song, and Yiyan Qi. ε-MemEvo: Adaptive cross-task memory transfer for LLM program evolution. *arXiv preprint arXiv:2608.12522*, 2026. URL https://arxiv.org/abs/2608.12522.

[22] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *arXiv preprint arXiv:2303.11366*, 2023. URL https://arxiv.org/abs/2303.11366.

[23] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. ExpeL: LLM agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024. URL https://arxiv.org/abs/2308.10144.

[24] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. In *International Conference on Learning Representations*, pages 12028–12068, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/3339f19c5fcee3ad74502947a32be9e6-Abstract-Conference.html.

[25] Haoran Ye, Jiarui Wang, Zhiguang Cao, Federico Berto, Chuanbo Hua, Haeyeon Kim, Jinkyoo Park, and Guojie Song. ReEvo: Large language models as hyper-heuristics with reflective evolution. In *Advances in Neural Information Processing Systems*, volume 37, pages 43571–43608, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/4ced59d480e07d290b6f29fc8798f195-Paper-Conference.pdf.

[26] Jiefeng Chen, Bhavana Dalvi Mishra, Jaehyun Nam, Rui Meng, Tomas Pfister, and Jinsung Yoon. Mars: Modular agent with reflective search for automated ai research. *arXiv preprint arXiv:2602.02660*, 2026.

[27] Shiyi Cao, Ziming Mao, Joseph E Gonzalez, and Ion Stoica. K-search: Llm kernel generation via co-evolving intrinsic world model. *arXiv preprint arXiv:2602.19128*, 2026.

[28] Anja Surina, Amin Mansouri, Lars Quaedvlieg, Amal Seddas, Maryna Viazovska, Emmanuel Abbe, and Caglar Gulcehre. Algorithm discovery with LLMs: Evolutionary search meets reinforcement learning. *arXiv preprint arXiv:2504.05108*, 2025. URL https://arxiv.org/abs/2504.05108.

[29] Peter Phan, Dhruv Agarwal, Kavitha Srinivas, Horst Samulowitz, Pavan Kapanipathi, and

Andrew McCallum. MiGrATe: Mixed-policy GRPO for adaptation at test-time. *arXiv preprint arXiv:2508.08641*, 2025. URL https://arxiv.org/abs/2508.08641.

[30] Thomas Hubert, Rishi Mehta, Laurent Sartran, Miklós Z. Horváth, Goran Žužić, Eric Wieser, Aja Huang, Julian Schrittwieser, Yannick Schroecker, Hussain Masoom, Ottavia Bertolli, Tom Zahavy, Amol Mandhane, Jessica Yung, Iuliya Beloshapka, Borja Ibarz, Vivek Veeriah, Lei Yu, Oliver Nash, Paul Lezeau, Salvatore Mercuri, Calle Sönne, Bhavik Mehta, Alex Davies, Daniel Zheng, Fabian Pedregosa, Yin Li, Ingrid von Glehn, Mark Rowland, Samuel Albanie, Ameya Velingker, Simon Schmitt, Edward Lockhart, Edward Hughes, Henryk Michalewski, Nicolas Sonnerat, Demis Hassabis, Pushmeet Kohli, and David Silver. Olympiad-level formal mathematical reasoning with reinforcement learning. *Nature*, 651(8106):607–613, 2026. doi: 10.1038/s41586-025-09833-y. URL https://www.nature.com/articles/s41586-025-09833-y.

[31] Qwen Team. Qwen3 technical report, 2025. URL https://arxiv.org/abs/2505.09388.

[32] OpenAI. gpt-oss-120b & gpt-oss-20b model card, 2025. URL https://arxiv.org/abs/2508.10925.

[33] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

[34] Máté Matolcsi and Carlos Vinuesa. Improved bounds on the supremum of autoconvolutions. *Journal of Mathematical Analysis and Applications*, 372(2):439–447.e2, 2010. doi: 10.1016/j.jmaa .2010.07.030. URL https://doi.org/10.1016/j.jmaa.2010.07.030.

[35] Richard C. Barnard and Stefan Steinerberger. Three convolution inequalities on the real line with connections to additive combinatorics. *Journal of Number Theory*, 207:42–55, 2020. doi: 10.1016/j.jnt.2019.07.001. URL https://arxiv.org/abs/1903.08731.

[36] Christopher Boyer and Zane Kun Li. An improved example for an autoconvolution inequality. *Experimental Mathematics*, pages 1–7, 2026. doi: 10.1080/10586458.2025.2607423. URL https://doi.org/10.1080/10586458.2025.2607423.

[37] Jan Kristian Haugland. The minimum overlap problem revisited. *arXiv preprint arXiv:1609.08000*, 2016. URL https://arxiv.org/abs/1609.08000.

[38] William P. Orrick, Bruce Solomon, Roland Dowdeswell, and Warren D. Smith. New lower bounds for the maximal determinant problem. *arXiv preprint arXiv:math/0304410*, 2003. URL https://arxiv.org/abs/math/0304410.

Appendix

A Task Definitions and Evaluation

We evaluate five mathematical construction tasks from TTT-Discover [1] and ThetaEvolve [6]: three autocorrelation inequalities (AC1–AC3), Erdős' minimum overlap problem, and the maximal determinant problem (Hadamard). The scores below retain the task's original objective direction; Section 3.1 uses a higher-is-better convention for $s(x)$.

A.1 First Autocorrelation Inequality (AC1)

For a nonzero, nonnegative step function f supported on $[-\frac{1}{4}, \frac{1}{4}]$, its autoconvolution is

$$(f * f)(t) = \int_{\mathbb{R}} f(x)\, f(t-x)\, dx.$$

AC1 minimizes the ratio [34, 35]

$$S_{\mathrm{AC1}}(f) = \frac{\|f * f\|_\infty}{\|f\|_1^2}. \tag{9}$$

For heights $a_0, \ldots, a_{n-1}$ on n equal-width intervals, this score is

$$S_{\mathrm{AC1}}(a) = \frac{2n \max_k (a * a)_k}{\left(\sum_{i=0}^{n-1} a_i\right)^2}, \tag{10}$$

where $a * a$ denotes discrete convolution.

A.2 Second Autocorrelation Inequality (AC2)

For the same class of step functions, AC2 maximizes the ratio [34, 36]

$$S_{\mathrm{AC2}}(f) = \frac{\|f * f\|_2^2}{\|f * f\|_1 \|f * f\|_\infty}. \tag{11}$$

We compute the squared L^2 norm by integrating over each linear segment of the autoconvolution. A segment of width Δ with endpoint values u and v contributes

$$\frac{\Delta}{3}\left(u^2 + uv + v^2\right).$$

A.3 Third Autocorrelation Inequality (AC3)

AC3 optimizes over real-valued step functions f satisfying

$$\mathrm{supp}(f) \subseteq \left[-\tfrac{1}{4}, \tfrac{1}{4}\right], \qquad \int_{-1/4}^{1/4} f(x)\, dx \neq 0.$$

The heights may be positive or negative. The objective is to minimize [6]

$$S_{\mathrm{AC3}}(f) = \frac{\|f * f\|_\infty}{\left(\int_{-1/4}^{1/4} f(x)\, dx\right)^2}. \tag{12}$$

For heights $a_0, \ldots, a_{n-1}$ on n equal-width intervals, the score is

$$S_{\mathrm{AC3}}(a) = \frac{2n \max_k |(a * a)_k|}{\left(\sum_{i=0}^{n-1} a_i\right)^2}. \tag{13}$$

A.4 Erdős' Minimum Overlap Problem

The continuous formulation [37] uses a step function h and its complement g:

$$h : [-1, 1] \to [0, 1], \qquad \int_{-1}^{1} h(x)\, dx = 1,$$
$$g(x) = 1 - h(x), \qquad x \in [-1, 1].$$

We extend both functions by zero outside this interval and minimize the maximum overlap

$$S_{\text{Erdős}}(h) = \sup_{t \in \mathbb{R}} \int_{\mathbb{R}} h(x) g(x + t)\, dx. \tag{14}$$

For heights $h_0, \ldots, h_{n-1}$ on n equal-width intervals, the constraints are

$$0 \le h_i \le 1 \quad (i = 0, \ldots, n - 1), \qquad \sum_{i=0}^{n-1} h_i = \frac{n}{2}.$$

The score is

$$S_{\text{Erdős}}(h) = \frac{2}{n} \max_{k=-(n-1),\ldots,n-1} \sum_{\substack{0 \le i < n \\ 0 \le i+k < n}} h_i(1 - h_{i+k}). \tag{15}$$

A.5 Maximal Determinant Problem (Hadamard)

The maximal determinant problem searches for a matrix with the largest absolute determinant [38]. We use the search space

$$H \in \{-1, +1\}^{n \times n}, \qquad n = 29.$$

We maximize the normalized score used in ThetaEvolve [6]:

$$S_{\text{Had}}(H) = \frac{|\det H|}{2^{28} \cdot 7^{12} \cdot 342}. \tag{16}$$

The evaluator computes the determinant using exact integer arithmetic.

Table 6: Evaluation metrics. Arrows indicate the direction of improvement. We compute best-found scores over all valid candidates evaluated up to the current search step.

Task	Score	Best-found
AC1 (↓)	Upper bound	Minimum
AC2 (↑)	Lower bound	Maximum
AC3 (↓)	Upper bound	Minimum
Erdős (↓)	Upper bound	Minimum
Hadamard (↑)	Normalized determinant	Maximum

B Experimental Setup

Within each framework, w/ TTME and w/o TTT use the same fixed-weight model, task initialization, evaluator, and primary candidate budget. Each run maintains its own search archive. Section C.1 defines the memory conditions.

B.1 ThetaEvolve Framework

We use the ThetaEvolve [6] inference framework on AC2, AC3, and the $n = 29$ Hadamard maximal-determinant problem, with DeepSeek-R1-0528-Qwen3-8B (Distill-Qwen3-8B) frozen throughout.

Search and generation. At each step, we select 32 parents from the MAP-Elites archive with islands and generate 16 primary candidates per parent. The w/ TTME setting allocates eight candidates to each generation branch. We construct prompts from the released expert context, parent program, and `SEARCH/REPLACE` rules, then update the archive with the evaluation results. Table 7 compares settings with the released inference-only configuration.

Calls for memory maintenance, strategy selection, and strategy adaptation add to the 33,280 primary candidate attempts.

Table 7: Settings for fixed-weight Distill-Qwen3-8B on AC2, AC3, and Hadamard. Original denotes ThetaEvolve's 65-step early inference-only setting [6]. Counts are per task and seed.

Setting	Original ThetaEvolve	Ours	
	w/o TTT	w/o TTT	w/ TTME
Search budget and archive			
Search steps	65	65	65
Parents × candidates per parent	$32 \times 16 = 512$	$32 \times 16 = 512$	$32 \times 16 = 512$
Primary candidates per run	33,280	33,280	33,280
Random seeds	42, 1234, 3407	3407	3407
Population / islands / archive	10,000 / 10 / 1,000	10,000 / 10 / 1,000	10,000 / 10 / 1,000
Generation			
Separate thinking-token cap	—	×	×
Output format	`SEARCH/REPLACE`	`SEARCH/REPLACE`	`SEARCH/REPLACE`
Evaluation			
Runtime / eval. timeout (s): AC2, Hadamard	300 / 350	300 / 350	300 / 350
Runtime / eval. timeout (s): AC3	60 / 70	60 / 70	60 / 70
Learning and memory			
Weight updates	×	×	×
Guideline Memory	×	×	✓
Strategy Memory	×	×	✓
Joint memory update interval, K (steps)	n/a	n/a	2

✓: enabled; ×: disabled; —: not reported; n/a: not applicable. The original paper also reports 100-step inference runs. Code reference: ThetaEvolve, commit `7c12898f`.

B.2 TTT-Discover Framework

We evaluate Qwen3-8B and gpt-oss-120b on AC1, AC2, and Erdős using TTT-Discover [1]. Both models use the search and evaluation settings below; Table 8 compares the gpt-oss-120b settings with the original configuration.

Search and generation. PUCT selects eight parents from the reuse buffer per step, and the model generates 64 candidates per parent. TTME allocates 32 to each generation branch. We retain the released task prompts, construction-state reuse, and verifier. Each rollout produces one complete Python program. The buffer admits up to two children per parent and retains the initial states.

Completion and evaluation. Both models use vLLM with a 32,768-token context, a 50-token reserve, temperature 1.0, top-p 1.0, and no top-k filtering. For gpt-oss-120b, the released sampler allows 26,000 tokens for the prompt and thinking, then continues in the final-answer channel when needed. Qwen3-8B uses its chat template with an open `<think>` prefix and the remaining context for the response, without forced final-answer continuation. Programs have a 1,000-second runtime budget and a 1,100-second evaluator timeout; failed programs receive no retry with error feedback. Each run has 25,600 primary candidate attempts. Memory updates, strategy selection, and adaptation require additional calls in w/ TTME.

Table 8: Settings for `gpt-oss-120b` on AC1, AC2, and Erdős. Original denotes TTT-Discover with reinforcement learning [1]; both of our conditions use frozen weights. Counts are per run.

Setting	Original TTT-Discover	Ours	
	w/ TTT	w/o TTT	w/ TTME
Search budget and reuse buffer			
Search steps	50	50	50
Parents × candidates per parent	$8 \times 64 = 512$	$8 \times 64 = 512$	$8 \times 64 = 512$
Primary candidates per run	25,600	25,600	25,600
Buffer capacity / PUCT coefficient c	1,000 / 1.0	1,000 / 1.0	1,000 / 1.0
Maximum admitted children per parent	2	2	2
Generation			
Context window (tokens)	32,768	32,768	32,768
Prompt + thinking limit (tokens)	26,000	26,000	26,000
Reasoning effort	high	high	high
Forced final-answer continuation	✓	✓	✓
Output format	Full program	Full program	Full program
Evaluation and serving			
Runtime / eval. timeout (s)	1,000 / 1,100	1,000 / 1,100	1,000 / 1,100
One retry with failure feedback	×	×	×
Model serving	Tinker	vLLM	vLLM
Learning and memory			
Weight updates	✓	×	×
LoRA rank	32	n/a	n/a
Guideline Memory	×	×	✓
Strategy Memory	×	×	✓
Joint memory update interval, K (steps)	n/a	n/a	2

✓: enabled; ×: disabled; n/a: not applicable. Code reference: TTT-Discover, commit `6c40e82d`.

C Memory Implementation Details

Each run starts with both memories empty. We use one modification instruction per parent ($N = 1$). Section D provides the prompt templates for distillation, consolidation, and adaptation.

C.1 Memory Overview

Full TTME enables both components in Table 9. Each component ablation removes one memory component and retains the other.

Table 9: Construction and use of the two memories. We update both from evaluated search outcomes every $K = 2$ steps.

Aspect	**Guideline Memory**	**Strategy Memory**
Input evidence	Valid outcomes and execution or validity failures collected across parents.	Evaluated parent–child pairs in which the child is valid and improves on the parent.
Model input	Outcome summaries and the current guidelines.	Parent and child code, their diff, and evaluation results.
Extracted content	An operational lesson: a trigger, a mechanism, and a concrete action.	An experience note: the algorithmic idea expressed by a change and its application conditions.
Memory update	Merge new lessons with current guidelines, combine duplicates, and retain at most 24.	Group notes into strategy families. Create an entry for a new family, or consolidate new notes with the existing entry.
Use during generation	Include current guidelines in the prompt for both generation branches.	Select a strategy for the parent, then adapt its entry into a modification instruction for Strategy-Guided Generation.

C.2 Memory Updates

We update both memories every $K = 2$ search steps using outcomes collected since the previous update.

Guideline Memory.

1. **Summarize recent outcomes.** The outcome summary covers recent attempts across parents, including valid results, their scores, and execution or validity failures.
2. **Distill guidelines.** The same fixed-weight base model reads the outcome summary and current guidelines, then proposes up to three lessons per update, at most one for failure avoidance. Each lesson specifies a trigger, explains the mechanism, and gives a concrete action. The parser extracts the lesson text from the response.
3. **Match and merge.** For each proposed lesson, we compute its Jaccard similarity to stored guidelines using lowercased word-token sets. A similarity of at least 0.5 counts as a match: we retain the stored text, increment its recurrence count, and refresh its last reaffirmed step. An unmatched lesson enters memory with count one and the current step.
4. **Remove stale entries and enforce capacity.** Guidelines expire when more than 20 search steps have passed since their last reaffirmation. Among the remaining entries, we retain at most 24, ranked by recurrence count with recency breaking ties.
5. **Include guidelines in generation prompts.** Both Direct Generation and Strategy-Guided Generation receive the retained guidelines as shared context. Each prompt displays up to 240 characters per guideline; memory stores the full lesson text.

Strategy Memory.

1. **Select valid improvement cases.** We select evaluated parent–child pairs in which the child passes the task verifier and improves on its parent. Both scores must be finite; improvement follows the task's objective direction, consistent with $s(\text{child}) > s(\text{parent})$ in Section 3.3.
2. **Extract experience notes.** The same fixed-weight base model reads the parent and child programs, their code diff, and evaluation results. It writes an experience note describing the algorithmic idea behind the change and its application conditions. Each note retains a reference to its source case; a change containing independent ideas can yield separate notes.
3. **Group notes into strategy families.** The model compares new notes with existing strategy entries and groups notes that share an underlying algorithmic idea into the same family. It creates a new family for a distinct idea and preserves implementation differences as variants within each family.
4. **Consolidate and store strategy entries.** For each family receiving new notes, the model combines those notes with the previous entry, if one exists, to update the description of the shared idea, application conditions, and implementation variants. We store the entry, its summary, and the supporting notes with source references. Families without new notes retain their existing entries.

C.3 Memory Hyperparameters

The limits on guideline count and text length are configurable hyperparameters. Table 10 lists our settings and candidate allocation.

Table 10: Memory hyperparameters and candidate allocation. We measure K in search steps; N is the number of modification instructions per parent. Character budgets apply to prompt-visible text.

Setting	TTT-Discover	ThetaEvolve
Joint memory update interval, K (steps)	2	2
Maximum stored guidelines	24	24
Maximum proposed guidelines per update	3	3
Guideline expiry window (search steps)	20	20
Guideline matching threshold (Jaccard)	0.5	0.5
Display budget per guideline (characters)	240	240
Modification instruction budget (characters)	700	700
Candidates per parent, B	64	16
Modification instructions per parent, N	1	1
Direct Generation candidates	32	8
Strategy-Guided Generation candidates	32	8
Candidates per modification instruction	32	8

D Prompts

The following prompts cover five memory operations. Section D.6 provides the input layouts supplied by the caller. Prompt wording retains the original terms *memo*, *method memory*, and *direction*; subsection titles follow the Method terminology. The example outputs were regenerated with `gpt-oss-120b` from historical AC1 and AC2 inputs. Examples 1–3 are excerpts, and Example 5 is complete. The guideline label is standardized, and `[...]` marks an omission.

D.1 Guideline Memory Update

The caller appends the current guidelines and a summary containing the objective direction, best score so far, representative successes across parents, and available failure summaries. The template below shows the base instruction.

Prompt 1

You maintain a compact Guideline Memory of transferable lessons for an iterative program-search over a numerical construction problem. From the step outcomes you are given, distill AT MOST 3 guidelines — fewer, including none, when the outcomes support fewer. A guideline must CHANGE what the program does; if the failing code already satisfies your action, the action is not the fix and repeating it will reproduce the failure.

Each guideline must be ONE line in exactly this form:

`Guideline <n>: <trigger> -> <mechanism> -> <action>`

Number the guidelines consecutively from 1. Guidelines may be scientific what-drives-score lessons or failure-avoidance lessons. At most one failure-avoidance guideline. The action must be a concrete, executable instruction (an exact quantity, function, or step), never a vague suggestion. Do not quote these instructions back; emit only the guideline lines inside a <guidelines>...</guidelines> block.

Example 1

Guideline 1: normal() got an unexpected keyword argument 'dtype' → `np.random.normal` does not accept a `dtype` parameter → replace each `np.random.normal(..., dtype=DTYPE)` with `np.random.normal(...).astype(DTYPE)` (or simply omit the `dtype` argument).

D.2 Strategy Memory: Experience Note Extraction

The caller appends the complete parent and child programs, their scores formatted to ten decimal places, and their code diff.

Prompt 2

You are reading one step of an automated program-search run on a numerical optimisation problem. A parent program was modified into a child program, and the child scored better.

Answer these four questions about THIS pair, briefly and concretely:

1. What substantive algorithmic change does the child make relative to the parent? Describe the change itself, not the names of functions.
2. Where in the program does the change take effect — which loop or phase?
3. Which part of the change is most plausibly connected to the score change?
4. Write ONE short memo (2-4 sentences) that would help someone making the next modification. Do not refer to any function or variable by name; a different implementation of the same idea must still match your memo.

Most parent-to-child modifications make ONE change. If this modification genuinely bundles two or three INDEPENDENT changes -- each would work, and could have been made, without the others -- answer all four questions for each change separately, and separate the answer blocks with a line containing only ===. Interlocking parts of one idea (a new stage plus the wiring that feeds it) are ONE change, not two.

Write plain prose. Do not use section headers. Do not speculate beyond what the two programs show.

Example 2

1. When multiple convolution entries share the current maximum, the child forms the descent direction by averaging the individual gradients for all those active indices, whereas the parent simply picks the single argmax entry.
2. This averaging is performed after the convolution is obtained and before the step-size scaling, inside the inner descent loop.

`[...]`

4. **Memo:** if the max convolution value occurs at several positions, combine their gradients (e.g. by averaging) to obtain the sub-gradient instead of using a single arbitrary index.

D.3 Strategy Memory: Strategy Entry Consolidation

The caller places an existing-entry or new-entry prefix before the shared instruction, then appends the corresponding inputs. The family label uses its quoted string representation.

Prompt 3

Write ONE method memory for this family, with these four parts:

1. MECHANISM - what the method actually does, in its own terms.
2. OBSERVED EXECUTION MODES - every distinct way it has been applied. Two modes are distinct when they differ in WHERE they run (before the loop, inside it, replacing the update, after it), in their SCHEDULE, in how the WORKING SET is chosen, or in their UPDATE / STATE-RESET semantics. They are the SAME mode when they differ only in wording or in ordinary numeric settings such as a step size or an iteration count.
3. WHEN EACH MODE SUITS A PROGRAM - the structural condition that makes a mode applicable to the program being improved.
4. WHAT MAY BE ADAPTED - which parts a per-parent rewrite is free to change.

Rules:

- Do NOT restate the same execution mode more than once, however differently the evidence worded it.
- Do NOT delete a genuinely distinct mode to be brief. There is no length limit; a simple family may need two sentences and a complex one a long paragraph.
- Do NOT invent a numeric threshold or hard rule that the evidence does not support.
- Output the memory only. No preamble, no JSON, no headings beyond the four parts above.

Example 3

OBSERVED EXECUTION MODES

1. **Post-up-sampling L-BFGS-B + final Adam polish** – The quasi-Newton refinement is called immediately after the two up-sampling loops (which already produced a high-resolution candidate) and is followed directly by a brief Adam polish; no further up-sampling occurs.
2. **Post-up-sampling + fine-tune L-BFGS-B → extra up-sampling → final Adam polish** – The refinement is inserted after the up-sampling and the preceding Adam fine-tuning stage, then its output is fed into an additional up-sampling step before the concluding Adam polish.

D.4 Strategy-Guided Search: Strategy Selection

In this template, `{k}` corresponds to the number of strategies per parent, N, rather than the update interval K. The caller requests the smaller of N and the number of available strategy families. The program and available-methods blocks are always included; other input blocks appear only when their context is available. Score history includes the current program and up to six recent ancestors, and each strategy summary contains at most 400 characters.

Prompt 4

You are steering an automated program-search run on a numerical optimisation problem. Below is the program you must improve next, how it and its recent ancestors scored, what has already been tried on this line of descent, and a list of METHODS this run has already found to work somewhere.

Select up to {k} methods from the list most likely to IMPROVE this program's score — including methods the program does not yet contain: similarity to what the program already does is not by itself a reason to choose or to avoid a method. The {k} must be DIFFERENT methods that lead to {k} different changes to this program: do not choose two methods that would make the same change. Take account of what this line of descent has already been given: repeating something that was tried here and did not move the score is usually worse than trying something this program has not seen. If several methods are equally plausible, prefer ones not already assigned to another group this step.

Return ONLY a JSON object, no prose before or after, with the ids in order of preference:

```
{"method_ids":  ["<id from the list>", "<id from the list>"]}
```

D.5 Strategy-Guided Search: Strategy Adaptation

The caller appends the parent program and the selected strategy's full entry. It omits the execution-summary block when unavailable and uses the strategy's short direction only when the full entry is empty.

Prompt 5

You are steering an automated program-search run on a numerical optimisation problem. A method has been chosen for the program below. You are given the accumulated memory of that method: what it does, every distinct way it has been applied, when each way suits a program, and what may be adapted.

Write a direction for THIS specific program: 2-4 sentences saying what to change and what would count as having done it. Pick the way of applying the method that suits the structure this program actually has -- where it runs, its schedule, how its working set is chosen -- and commit to that one choice rather than listing alternatives.

The direction is the only thing the next author will see. Do not name any function or variable, do not mention scores or this run, and do not mention that a method list or a memory exists. Output the direction only -- plain prose, no preamble, no JSON, no headings.

Example 5

Insert a bounded L-BFGS-B optimisation step immediately after the second up-sampling and its fine-tuning, using the exact gradient of the negative lower-bound and enforcing non-negativity box constraints, and run it for a modest iteration budget (e.g., a few hundred steps) until the convergence tolerance is met. Then apply a brief Adam polishing pass (a few thousand steps) to re-apply the projection and clean any residual noise. The program is complete when the final candidate passed to the scorer is the result of this L-BFGS-B followed by the short Adam refinement.

D.6 Input Templates

The caller fills the placeholders below and appends each input block to its corresponding fixed instruction. Prompt 3 also receives a prefix before the instruction; its two variants are shown below.

Inputs for Prompt 1

````
## CURRENT playbook
{current_guideline_memory}
## THIS STEP
{step_outcome_summary}
Emit the updated lessons now.
````

Inputs for Prompt 2

````
## PARENT PROGRAM (score {parent_score})
```python
{parent_program}
```
## CHILD PROGRAM (score {child_score}, better)
```python
{child_program}
```
## DIFF (parent -> child)
```diff
{parent_to_child_diff}
```
````

Inputs for Prompt 3

Existing entry: prefix before Prompt 3

```
Here is the current method memory for the family {family_label_repr}, and the notes that
arrived at this boundary.  Rewrite the memory so it covers both.  Keep every execution
mode already in it unless a new note shows it was wrong.
```

Inputs after Prompt 3

```
## CURRENT MEMORY
{current_strategy_entry}
### new note 1
{experience_note_1}
### new note 2
{experience_note_2}
...
```

New entry: alternative prefix before Prompt 3

```
You are starting the method memory for the family {family_label_repr} from nothing.
```

For a new entry, append only the numbered new notes and omit the `CURRENT MEMORY` block. The number of notes varies.

Inputs for Prompt 4

```
## PROGRAM TO IMPROVE
{parent_program}
## WHAT THIS PROGRAM ACTUALLY EXECUTED (self-reported)
{self_reported_execution_summary}
```

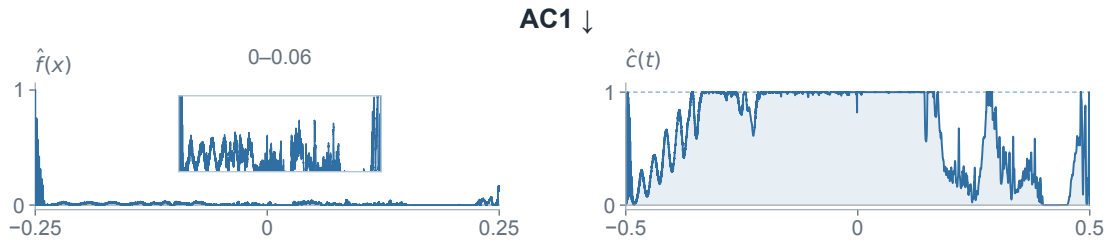
```
## SCORE OF THIS PROGRAM AND ITS RECENT ANCESTORS
{Higher_or_LOWER} is better.  {scores_most_recent_first} (most recent first)
## WHAT THIS LINE OF DESCENT HAS ALREADY BEEN GIVEN
{lineage_assignment_history}
## WHAT HAS GONE WRONG BEFORE
{memory_context}
## METHODS ALREADY ASSIGNED TO OTHER GROUPS THIS STEP
- {already_assigned_method_1}
- {already_assigned_method_2}
...
## METHODS THIS RUN HAS FOUND
- id `{method_id_1}`:  {strategy_summary_1}
- id `{method_id_2}`:  {strategy_summary_2}
...
```

Inputs for Prompt 5

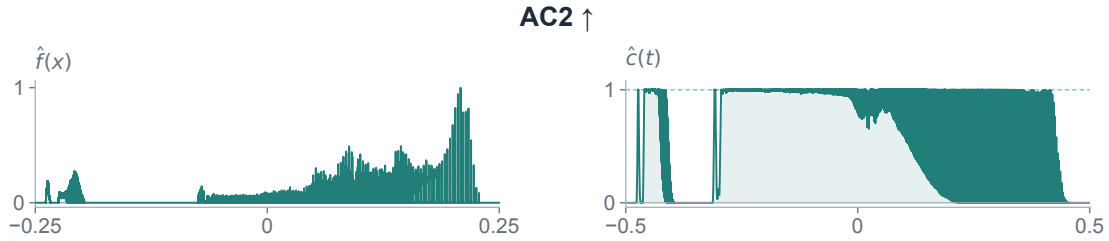
```
## PROGRAM TO IMPROVE
{parent_program}
## WHAT THIS PROGRAM ACTUALLY EXECUTED (self-reported)
{self_reported_execution_summary}
## THE CHOSEN METHOD'S ACCUMULATED MEMORY
{strategy_label_or_id}
{full_strategy_entry_or_fallback_direction}
```

E Discovered Constructions

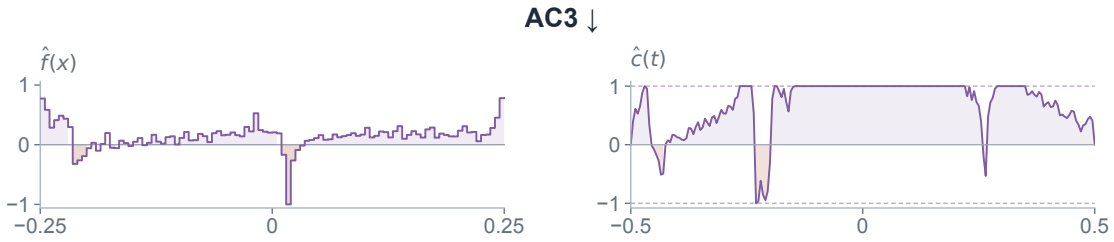
Figure 6 shows constructions for the five tasks defined in Appendix A.



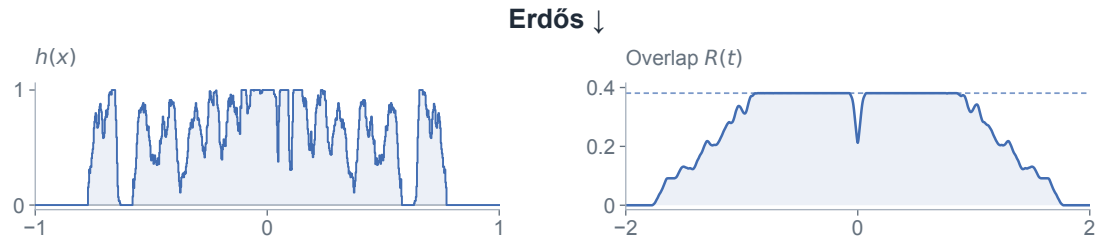
(a) *gpt-oss-120b* produced a 32 000-bin AC1 construction with an objective value of 1.502745082. The plots show the normalized step function $\hat{f}(x)$ (left) and autoconvolution $\hat{c}(t)$ (right).



(b) *gpt-oss-120b* produced a 65 536-bin AC2 construction with an objective value of 0.960784. The plots show the normalized step function $\hat{f}(x)$ (left) and autoconvolution $\hat{c}(t)$ (right).

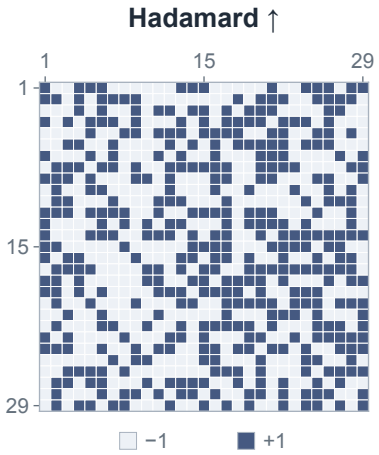


(c) *Distill-Qwen3-8B* produced a 100-bin AC3 construction with an objective value of 1.464224300. The plots show the normalized step function $\hat{f}(x)$ (left) and autoconvolution $\hat{c}(t)$ (right).



(d) *gpt-oss-120b* produced the 801-bin Erdős construction with a maximum overlap of 0.380861541. The left plot shows its density $h(x)$ on $[-1, 1]$, and the right plot shows the corresponding overlap $R(t)$.

Figure 6: Discovered constructions for AC1–AC3 and Erdős. The AC1 inset shows normalized heights 0–0.06. Arrows indicate the direction of improvement.



(e) *Distill-Qwen3-8B*, with memory enabled, produced the 29×29 sign matrix H, attaining an objective value of 0.579938205. Dark and light cells denote $+1$ and -1, respectively.

Figure 6: Discovered constructions (continued). Panel (e) preserves the original row and column order of the Hadamard construction.